

Matriz de Optimización de Costos

Qué modelo se usa por tarea, por qué, y cuánto se ahorra realmente — Ecosistema de Atención al Cliente con Memoria (Estudio Contable)

Este documento no se limita a “limitar tokens”: es una decisión de arquitectura. Se evalúan las tareas del ecosistema, se elige un modelo por tarea según su naturaleza, y se calcula el ahorro real de aplicar la Batch API y la selección de tier de modelo. Todos los montos son estimaciones para un volumen ilustrativo de **150 consultas/mes** (estudio contable pequeño/mediano); se explicita cada supuesto de tokens usado en el cálculo. **Nota:** las tarifas de IA cambian con frecuencia; estos valores son de referencia (septiembre 2026) y conviene reverificarlos en la página de pricing oficial antes de la entrega.

Tarifas vigentes usadas en este cálculo (Gemini API, por millón de tokens, precio de lista)

Modelo	Input	Output	Mínimo cacheable	Batch API
Gemini 3.6 Flash	\$1.50	\$7.50	4.096 tokens	-50%
Gemini 3.1 Flash-Lite	\$0.25	\$1.50	No confirmado*	-50%
OpenAI GPT-4o-mini (ref.)	\$0.15	\$0.60	-	-50%
Claude Sonnet 5 (ref., proveedor descartado)	\$2.00	\$10.00	1.024 tokens	-50%

* La documentación oficial de Gemini confirma el mínimo cacheable para los modelos Flash y Pro (4.096 y 2.048 tokens según versión) pero no lo publica para Flash-Lite al momento de escribir esto; no se usa caching en la Tarea C de todas formas (se optimiza vía Batch API), así que no afecta ningún cálculo de este documento.

1. Qué modelo por tarea, y por qué

Tarea B — Clasificar la consulta y redactar la respuesta (nodo principal del flujo)

Modelo elegido: **Gemini 3.6 Flash**. Es la tarea que exige lectura densa (historial del cliente + email nuevo) y razonamiento sobre tono y contenido antes de que un humano lo apruebe; un modelo Flash-Lite es arriesgado para un texto que sale a nombre del estudio. **Importante:** a diferencia de Anthropic, el caching implícito de Gemini 3.6 Flash recién se activa a partir de 4.096 tokens de prefijo repetido. El bloque fijo de instrucciones de este sistema (~600 tokens) está muy por debajo de ese mínimo, así que **no hay ahorro de caching disponible en esta tarea a esta escala** — se documenta igual en el Manual Operativo de Datos (doc. 2) porque cambia cómo se calcula el costo real.

Tarea C — Resumen mensual de KPIs para el Dashboard de Control (proceso masivo, no urgente)

Modelo elegido: **Gemini 3.1 Flash-Lite, vía Batch API**. Una vez al mes se reprocesan las ~150 consultas del período para generar el resumen de KPIs y detectar anomalías. No hay usuario esperando una respuesta en tiempo real, así que corre en modo batch con 50% de descuento; al ser clasificación/agregación (no redacción

creativa), Flash-Lite alcanza sin pagar la tarifa de Flash.

Tarea A — Downgrade de Tarea B a Flash-Lite (evaluada, no implementada)

Se evaluó usar Gemini 3.1 Flash-Lite también para la Tarea B en lugar de Flash: el ahorro sería del **82%** sobre esa tarea. Se descarta por ahora: Flash-Lite es un modelo más chico, pensado para clasificación/extracción de alto volumen, y el riesgo de una redacción de tono inconsistente en una comunicación real a un cliente pesa más que el ahorro (montos absolutos bajos a este volumen). Queda documentado como el primer cambio a evaluar con más volumen o con un paso de revisión de calidad adicional antes del HITL.

Se evaluó también OpenAI GPT-4o-mini como alternativa ultra económica (ver tabla de tarifas); se descarta por la misma razón que Flash-Lite más el costo de mantener credenciales y un esquema de salida JSON de un segundo proveedor (ver Documento 4, Seguridad y Resiliencia).

2. Cálculo de costos y ahorro

Tarea B: Gemini 3.6 Flash — sin caching disponible

Supuestos: bloque fijo de 600 tokens + contexto variable (cliente + historial + email) de 500 tokens = 1.100 tokens de input, 150 de output, 150 ejecuciones/mes. Con Flash y sin caching, este es el costo real (no hay un escenario “con optimización” distinto, porque la única optimización de esta tarea sería bajar de modelo — evaluada arriba y descartada).

Escenario	Costo input	Costo output	Total/mes
Gemini 3.6 Flash (real, sin caching posible)	\$0.2475	\$0.1688	\$0.4163
Hipotético: mismo prompt en Flash-Lite (no adoptado)	\$0.0413	\$0.0338	\$0.0750

Diferencia entre ambos: 82.0% — el techo de ahorro posible en esta tarea si se aceptara el riesgo de calidad.

Tarea C: Gemini 3.1 Flash-Lite, estándar vs. Batch API

Supuestos: 250 tokens de input y 80 de output por registro reprocesado, 150 registros/mes, 1 corrida mensual.

Escenario	Total/mes
Estándar (tiempo real)	\$0.0274
Batch API (-50%)	\$0.0137

Ahorro por Batch API: 50% sobre esta tarea — la única optimización de costos que sí se implementa en la versión actual.

Total del ecosistema (tareas B + C)

Escenario	Total/mes
Sin ninguna optimización (B a precio de lista + C en tiempo real)	\$0.4436
Con Batch API en la Tarea C (Tarea B no tiene palanca de caching)	\$0.4299
Ahorro combinado	3.1%

El ahorro combinado (≈3%) es bajo comparado con lo que daría Prompt Caching en Claude (≈29%) para un prompt del mismo tamaño, y es la conclusión más importante de este documento: **en Gemini, con prompts de este tamaño, la palanca de costos real no es el caching sino la elección de tier de modelo por tarea** (Flash vs. Flash-Lite) y el uso de Batch API en todo lo que no sea tiempo real. Si el estudio creciera y el bloque fijo de instrucciones superara los 4.096 tokens (por ejemplo, agregando un manual de tono completo y ejemplos de respuestas aprobadas como few-shot), ahí el caching de Gemini empezaría a rendir y valdría la pena rehacer este cálculo.

Referencia cruzada: Gemini vs. Claude a precio de lista

Gemini 3.6 Flash sin ninguna optimización (\$0.4163/mes en la Tarea B) sale **25% más barato** que Claude Sonnet 5 sin optimizar (\$0.5550/mes para el mismo volumen y tamaño de prompt), pero **5.5% más caro** que Claude Sonnet 5 con Prompt Caching activo (\$0.3944/mes). Es decir: para este caso de uso puntual, Gemini gana en tarifa base pero pierde la ventaja de caching que sí tiene Claude a este tamaño de prompt.

Nota de proveedor único: el sistema usa exclusivamente la API de Google (Gemini) para todas las llamadas de IA del flujo. Las filas de OpenAI y Anthropic en la tabla de tarifas se incluyen solo como referencia comparativa.